



University of Asia Pacific
Department of Computer Science and Engineering
CSE 402: Operating System Lab

LAB REPORT

Submitted by: Shad Bin Moshiur

Student ID: 23101143

Section: C-2

Lab task: 2

Submitted to:

Atia Rahman Orthi

Lecturer,

Department of Computer Science and Engineering

Date of Submission: 03/08/2026

LAB 2 REPORT

First Come First Serve (FCFS) CPU Scheduling

1. Problem Statement

Implement First Come First Serve (FCFS) CPU Scheduling for a set of 5 processes. Each process has an Arrival Time (AT) and a Burst Time (BT), as given in the table below. The task is to calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for every process, along with the Average TAT and Average WT, and to represent the process execution sequence.

P id	AT	BT
P0	3	1
P1	5	3
P2	2	2
P3	1	2
P4	6	3

In FCFS scheduling, the process that arrives first is executed first (non-preemptive). If two processes arrive at the same time, the one with the smaller Process ID / lower arrival index is generally given priority.

2. Source Code (Python)

The processes are first sorted by Arrival Time. The CPU timeline is then simulated sequentially, keeping the CPU idle if the next process hasn't arrived yet.

```
index = [  
    ["P0", 3, 1],  
    ["P1", 5, 3],  
    ["P2", 2, 2],  
    ["P3", 1, 2],  
    ["P4", 6, 3]  
]  
  
index.sort(key=lambda x: x[1])  
  
current_time = 0  
  
total_tat = 0  
total_wt = 0  
  
print("PID\tAT\tBT\tCT\tTAT\tWT")
```

```

for p in index:
    pid, at, bt = p

    if current_time < at:
        current_time = at

    current_time += bt
    ct = current_time

    tat = ct - at
    wt = tat - bt

    total_tat += tat
    total_wt += wt

    print(f"{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")

avg_tat = total_tat / len(index)
avg_wt = total_wt / len(index)

print("\nAverage TAT =", round(avg_tat, 2))
print("Average WT =", round(avg_wt, 2))

```

3. Output

PID	AT	BT	CT	TAT	WT
P3	1	2	3	2	0
P2	2	2	5	3	1
P0	3	1	6	3	2
P1	5	3	9	4	1
P4	6	3	12	6	3

Average TAT = 3.6
Average WT = 1.4

Result Table:

PID	AT	BT	CT	TAT	WT
P3	1	2	3	2	0
P2	2	2	5	3	1
P0	3	1	6	3	2
P1	5	3	9	4	1
P4	6	3	12	6	3

Average TAT = 3.6 , Average WT = 1.4

4. Process Execution Sequence (Gantt Chart)

The CPU executes processes strictly in the order of arrival:

idle	p3	p2	p0	p1	p4
0	1	3 5	6	9	12

Execution order: P3 → P2 → P0 → P1 → P4

5. Discussion

While implementing FCFS scheduling, the main challenge was handling the case where the CPU becomes idle because the next process (by arrival time) hasn't arrived yet. This was solved by checking if $\text{current_time} < \text{arrival_time}$ and, if so, advancing current_time to the arrival time before adding the burst time.

Another important point was that FCFS scheduling depends entirely on sorting by Arrival Time, not on the original Process ID order given in the table. After sorting by AT, the actual execution order became P3, P2, P0, P1, P4, which is different from the P0–P4 order listed in the problem statement. Getting this sorting step right was essential, otherwise the CT/TAT/WT values would all be incorrect.

I also understood more clearly why FCFS suffers from the 'convoy effect' — since P1 and P4 both have relatively large burst times, they push back the completion time of later processes, increasing the average waiting time compared to what a shorter-job-first approach could achieve.

6. Conclusion

The FCFS CPU Scheduling algorithm was successfully implemented and verified using the given 5-process dataset. The average Turnaround Time was found to be 3.6 units and the average Waiting Time was 1.4 units. FCFS is simple to implement since it only requires sorting by arrival time and simulating the CPU timeline sequentially, but it does not always give the most efficient scheduling in terms of average waiting time, especially when processes with large burst times arrive early. This lab helped in understanding both the implementation logic and the practical limitations of the FCFS scheduling algorithm.

[Github Link:](https://github.com/Moshiur1143/Operating-System-Repository-23101143-/tree/main/Lab-02-FCFS-Implementation)

<https://github.com/Moshiur1143/Operating-System-Repository-23101143-/tree/main/Lab-02-FCFS-Implementation>